

The time-impact optimal trajectory planning for multi-joint robot based on GA-PSO

Rao Yao

Shanghai University of
Engineering Science
School of Mechanical and
Automotive Engineering
Shanghai, China
yaorao@sues.edu.cn
<https://orcid.org/0009-0001-3594-3426>

Jiaran Wang

Shanghai University of
Engineering Science
School of Mechanical and
Automotive Engineering
Shanghai, China
1106668776@qq.com

Yahui Lv

Aviation Industry Corporation of
China
Shanghai Aero Measurement-
Controlling Research Institute
Shanghai, China
2022590004@qq.com

Dazhong Wang*

Shanghai University of
Engineering Science
School of Mechanical and
Automotive Engineering
Shanghai, China
<https://orcid.org/0000-0003-4658-0373>, wdzh1689@126.com
*Corresponding author

Abstract—In order to simplify robot control and reduce the programming difficulty of impact-optimized trajectories, this paper studies the application of gesture recognition in industrial robot trajectory planning. Meanwhile for reducing the vibration and mechanical wear caused by excessive robot impact, it is proposed to use an improved GA-PSO algorithm for time-impact optimal trajectory planning, that is, to use adaptive inertia weight and crossover mutation probability. The improved algorithm has the characteristics of fast convergence in the early stage of iteration and not easy to fall into local optimal solution in the later stage of iteration. In the trajectory planning, a 5-5-5 interpolation polynomial is used to ensure the continuity of the trajectory planning speed, acceleration, and impact. Under the control of the set fitness function, the search of the improved GA-PSO algorithm is used to simulate the results. The experimental results show that the optimized trajectory significantly reduces the impact simultaneously.

Keywords—improved genetic particle swarm optimization algorithm; gesture recognition; trajectory planning

I. INTRODUCTION

With the development of robotics technology and human-machine interaction technology, applying gesture recognition to the path planning of industrial robots has become feasible. The traditional process of robot teaching is complex and cumbersome, requiring high demands on operators. Obtaining robot trajectories based on natural gesture interaction has the advantages of simplicity, low learning cost, and convenience of use, and can reduce the requirements on operators. For example, Leap Motion sensors is applied to obtain gesture coordinate dynamics for moving robots, achieving real-time data such as moving, grasping, and placing [1].

However, the initial trajectory of the gesture has a large degree of discreteness, and the resulting trajectory has a slow operation time and large impact. In order to improve the work efficiency and stability of industrial robots, it is possible to perform time-impact optimal trajectory planning on the operation trajectory obtained based on gestures. For example, in reference [2], Madridano et al. used a genetic algorithm for time-optimal trajectory planning, and obtained the optimal time trajectory. However, this method considered the impact, but it can be seen from the results that the acceleration has a large mutation, which not only affects the working accuracy of the

robot, but also shortens the service life [3]. To solve this problem, reference [4] carried out optimal impact trajectory planning, time-optimal and smooth trajectory planning for robot manipulators, and reference [5] used PSO algorithm and clustering algorithm to plan in joint space, and obtained the optimal impact trajectory by optimizing the nodes inserted in the trajectory. Liu [6] applied GA-PSO algorithm in work subspace division to achieve optimized manipulator trajectory. Yu et al. [7] proposed using GA-PSO algorithm to improve the search ability of simultaneous time-path-optimal trajectory planning for manipulator.

Furthermore, this paper proposes the use of an improved GA-PSO algorithm for time-impact optimal trajectory planning for large discreteness gesture data, improving the gesture trajectory to better adapt to industrial robots. The algorithm uses adaptive inertia weight and crossover mutation probability, and converges faster than the GA-PSO algorithm. Using a 5-5-5 interpolation polynomial to ensure the continuity of the robot's joint space speed, acceleration, and jerk, and verified on the Epson robot, the robot trajectory significantly reduced the impact without reducing the time.

II. GESTURE COORDINATE ACQUISITION

Due to the difference between the built-in coordinate system of Leap Motion and the robot, this paper determines the robot trajectory based on the relative coordinates of gestures. The robot's origin is used as the starting point for relative motion based on gestures, and the following formula is used to obtain the relative displacement of the robot in the x and y directions.

$$\begin{cases} X_t = \frac{L * (X_{t+1} - X_t)}{\max X - \min X} \\ Y_t = \frac{L * (Y_{t+1} - Y_t)}{\max Y - \min Y} \end{cases} \quad (1)$$

where X_i and Y_i are the coordinates before modification, X_t and Y_t are the coordinates after modification, L and W are the length and width of the template, respectively. The robot coordinates are obtained according to the coordinate range ratio. Taking the EPsonC3 robot as the model, using the Matlab robot toolbox for simulation (reduced by 10:1 relative to the original model), the modified coordinates are inversely solved to obtain the continuous coordinates of the six-axis angles. The robot

model can be driven to move along the gesture trajectory, and the result path is shown in Figure 1(c) below.

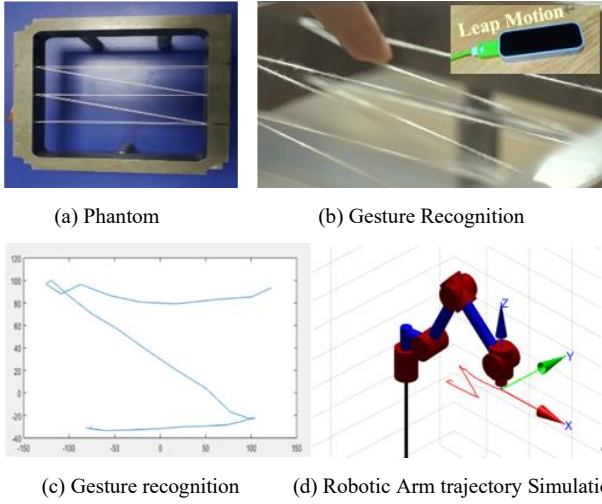


Fig. 1 Gesture recognition and robotic arm simulation system

In order for the gesture trajectory to be applicable to actual production, the instability of the hardware equipment and the discontinuity of the operator need to be eliminated. In general, filtering can be performed on information with noise to filter out frames with errors to ensure the accuracy of gesture control. With the development of robot vision, modifying the original trajectory by processing images obtained by the robot and feeding back to the servo controller can reproduce complex trajectories. After the gesture coordinates are renormalized, the obtained coordinates can be directly applied to robot operation, and the robot's running trajectory along the gesture path can be obtained.

For industrial production tasks similar to Figure 1(d), high frequency and speed are required. If the trajectory is not optimized, it will cause vibration and wear. This paper uses gesture recognition to obtain coordinates, and selects key points according to production needs to perform time-impact optimal trajectory planning on key points in the trajectory, and finally obtains a trajectory that can pass through key points at high speed and low impact. Taking the starting point, end point, and two turning points as the four key points for trajectory planning, the time-impact optimal trajectory based on gesture trajectory is obtained.

III. GA-PSO ALGORITHM

Natural gesture recognition can be applied to the automation programming of robots. In addition to fully applying the gesture path in the previous section, sometimes it is necessary to plan the key path of the robot based on gestures and perform optimal trajectory planning on it for point-to-point trajectory planning of industrial robots. Without specific path requirements, trajectory can usually be obtained based on interpolation algorithms according to the starting and ending positions [1].

A. Construction of Interpolation Function

Considering the optimal time and impact, this paper uses a 5-5-5 interpolation polynomial curve. Due to previous using

a 3-5-3 polynomial interpolation curve for time-optimal trajectory planning, the impact at the connection between the two time segments was not considered. This paper constructs a 5-5-5 interpolation polynomial, where the fifth-order polynomial can ensure the continuity of acceleration at the connection of trajectories. The general form of the polynomial is as follows:

$$X_{i1} = a_{i6} + a_{i5}t_1 + a_{i4}t_1^2 + a_{i3}t_1^3 + a_{i2}t_1^4 + a_{i1}t_1^5 \quad (2)$$

$$X_{i2} = a_{i12} + a_{i11}t_2 + a_{i10}t_2^2 + a_{i9}t_2^3 + a_{i8}t_2^4 + a_{i7}t_2^5 \quad (3)$$

$$X_{i3} = a_{i18} + a_{i17}t_3 + a_{i16}t_3^2 + a_{i15}t_3^3 + a_{i14}t_3^4 + a_{i13}t_3^5 \quad (4)$$

where X_{ij} is the angle of the j th segment of the i th axis, the starting point and end point angles, angular velocity, and angular acceleration are known in point-to-point trajectory planning, and the path points X_{i20} and X_{i30} are known. The coefficients of the polynomial can be obtained according to the time t_1 , t_2 , and t_3 , and the velocity, acceleration, and jerk in this segment of time can be calculated by differentiating the five-order polynomial of angular displacement. According to the correspondence of each time segment, the following matrix and coefficient a relationship can be obtained, as shown in Equations 5-7. Where a is the coefficient of the fifth-order polynomial, r_0 , r_1 , r_2 , r_3 are the angles at the starting point, the first point, the second point, and the end point of the axis, respectively. Due to the characteristics of the matrix, the invertibility of the matrix A needs to be considered in each step of operation.

B. GA-PSO Algorithm

The particle swarm optimization (PSO) algorithm is a global evolutionary optimization algorithm based on swarm intelligence, which simulates the foraging behavior of birds. The individuals in the population cooperate with each other to seek the optimal solution. Each potential solution to an optimization problem is a particle in the search space. Each particle has two characteristics: position and velocity. The fitness of a particle is the value of the objective function corresponding to its position coordinates, which measures the quality of the particle. In each iteration, particles update themselves by tracking two extremes. The first extreme is the best solution found by the particle itself, called individual best (pBest). The other extreme is the best solution found by the entire population, which is the global best (gBest). When finding these two best values, particles update their velocity and position according to the following formulas:

$$v_{t+1} = W * v_t + C_1 * rand * (pbest_t - x_t) + C_2 * rand * (gbest_t - x_t) \quad (5)$$

$$x_{t+1} = x_t + v_{t+1} \quad (6)$$

where W is the inertia weight, which is the influence of the previous particle velocity on the current particle velocity. Kennedy proposed it as a contraction factor to reduce particle oscillation with iteration and accelerate convergence speed. For the inertia weight, this paper uses a time-varying weight that adapts to the number of iterations:

$$W = 0.3 \left(\frac{i}{T}\right)^k$$

where i is the current iteration number, T is the total number of iterations. When $k > 0$, the square of the ratio is a

concave curve from 0 to 1, which means that the inertia weight W is a convex curve from 1 to 0.3. Therefore, the inertia weight is greatly affected by the previous particle velocity in the early stage of iteration and decreases rapidly in the later stage of iteration, which can prevent the particle swarm from converging too early to a local optimal solution and does not affect the search accuracy in the later stage of iteration.

To find the optimal time-impact trajectory planning, a significant method was to assign corresponding weights to each optimization objective, converting them into single objectives. By adjusting the weights of the two optimization objectives of time and impact, significant a trajectory that ensured a small impact could be obtained while improving speed and greatly reducing the motion time. Furtherly, time, acceleration, and impact were mixed to optimize the trajectory. This paper weights the difference between the particle's jerk and the maximum jerk constraint and sums it with the total time as the fitness function, as shown in Equation (7):

$$f_1 = W_t(t_1 + t_2 + t_3) + W_v * |V_{\max} - \max |V_i| + W_a * |a_{\max} - \max |a_i| + W_j * |j_{\max} - \max |j_i| \quad (7)$$

While when the global optimal solution of the current iteration does not meet the constraints, the single-dimensional time segment needs to be adjusted and modified to satisfy the constraints. The shortest time that satisfies the constraints is taken as the global optimal solution for the next iteration, and its modification function is shown in Equation (8):

$$f_2 = W_t * (t_1 + t_2 + t_3) + W_v * |\max |V_i| + W_a * |\max |a_i| + W_j * |\max |j_i| \quad (8)$$

Particle swarm algorithm has the advantages of fast search speed and high accuracy, but it is prone to fall into local optimal solutions. The genetic algorithm (GA) uses selection, crossover, and mutation operations to generate offspring. After adding the particle swarm algorithm, the particle swarm can well preserve the diversity of particles, effectively avoiding falling into local optimal solutions. This paper embeds the genetic algorithm's crossover and mutation operations after the particle swarm optimization algorithm to preserve the diversity of the particle swarm. In order not to affect the optimization performance of the particle swarm algorithm in the early stage of iteration, an adaptive crossover and mutation probability that increases with the number of iterations is used.

$$p_i = \begin{cases} 0.1 & i \leq \frac{T}{3} \\ p * (\frac{i}{T})^k & i > \frac{T}{3} \end{cases} \quad (9)$$

The variable p represents the crossover or mutation probability. In the experimental process, the particle swarm algorithm generally rapidly converges to the optimal solution around one-third of the iterations. Therefore, before this point, a relatively small crossover and mutation probability is taken, gradually increasing it thereafter to the maximum set crossover and mutation probabilities.

The specific algorithm process is as follows:

1) Initialize particles; set the number of particles and their dimensions, and represent particles with three segments of time t .

2) Calculate the fitness of particles; according to the three segments of time, substitute into Equations (4) to obtain the coefficient vector a . Then substitute into Equations (1)-(3) to obtain the maximum speed, maximum acceleration, and maximum jerk in three segments of time, and obtain individual best pBest and global best gBest.

3) Judge whether the global optimal solution exceeds the maximum constraint. If it does not exceed the maximum constraint, iterate over three segments of time according to Equations (7), through Equations (5)-(6) to obtain new particles. Perform crossover and mutation operations on the new particles, and compare the fitness values of the results obtained by the new particle swarm with the original particle swarm to select the particle swarm obtained in the current iteration with the lower fitness value; if it exceeds the maximum constraint, use Equation (8) as the fitness function to iteratively select a new global optimal solution that meets the constraints for the dimension of the time segment that exceeds the constraints.

4) Judge whether the termination condition is met. If not, return to 2.

5) Taking a robot that satisfies the pipper criterion as an example, the positions of the end of the first three axes can determine the end position of the manipulator. According to whether there is an attitude change in the path planning, choose the optimization of the six-axis angle or three-axis angle, and respectively obtain the optimal solution for the six axes or three axes that need to be solved. Select the maximum value of each segment of time as the final optimal time, and substitute the final three segments of time into the trajectory point requirements of each axis to obtain the optimal trajectory.

IV. SIMULATION EXPERIMENT

This study conducts simulation verification using the EpsonC4 robot to find the optimal time-impact trajectory that satisfies the robot's motion range and constraints. Specific constraints are shown in Table 1.

TABLE I. JOINT SPACE COORDINATE POINTS

Joints	Parameters	
	Maximum Working Range	Maximum Working Speed
J1	$\pm 180^\circ$	450 degrees/second
J2	$-160 \sim 65^\circ$	450 degrees/second
J3	$-51 \sim 225^\circ$	513 degrees/second
J4	$\pm 200^\circ$	555 degrees/second
J5	$\pm 135^\circ$	555 degrees/second
J6	$\pm 360^\circ$	720 degrees/second

Taking the trajectory turning points in the first section as an example, coordinates (241, -37, 0), (385, -34, 0), (200, 38, 0), (400, 36, 0) are selected. Starting from the first point and ending at the last point, with the posture coordinates unchanged, the optimization of the six-axis joint angles is performed. The inverse kinematics are used to solve the six-axis joint angles under the four-point pose states of the robot model in Figure 1, where the posture of the fourth axis mainly controls the Z-axis

rotation, and since this rule does not involve Z-axis rotation, it is zero.

TABLE II. DIMENSION PARAMETERS

Joints	Parameters			
	Path Point 1	Path Point 2	Path Point 3	Path Point 4
J1	-8.786°	-4.953°	10.739°	5.139°
J2	0.650°	26.202°	11.566°	-28.647°
J3	-0.643°	35.271°	-9.631°	39.286°
J4	0°	0°	0°	0°
J5	-0.006°	-9.068°	-1.935°	-10.638°
J6	8.786°	4.953°	-10.739°	-5.139°

SO algorithm, GA-PSO algorithm, and improved GA-PSO algorithm are used to solve this problem, with 30 particles each. The PSO algorithm sets the inertia weight $W=0.7$; the GA-PSO algorithm uses a fixed inertia weight of 0.7, and the crossover and mutation probabilities are 0.7 and 0.5, respectively; the improved GA-PSO algorithm sets 30 particles, with an initial crossover and mutation probability of 0.1, which is increased after meeting the conditions, with a maximum crossover probability of 0.7 and a maximum mutation probability of 0.5. The inertia weight is set according to Equation 9, with $k=1.5$. The optimal result of the first axis under 30 iterations is shown in Figure 2.

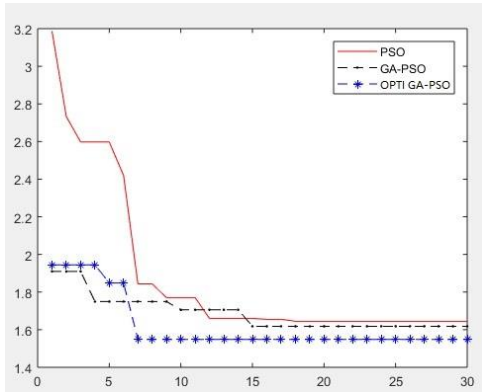
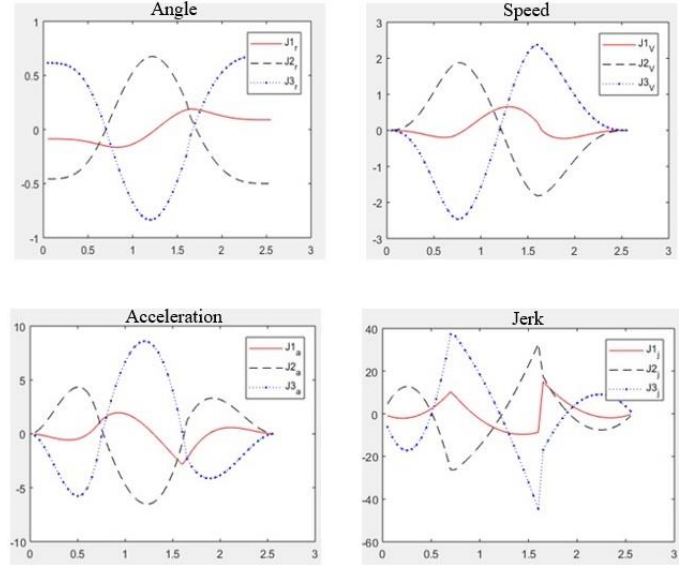


Fig. 2 Iterative Convergence Effect

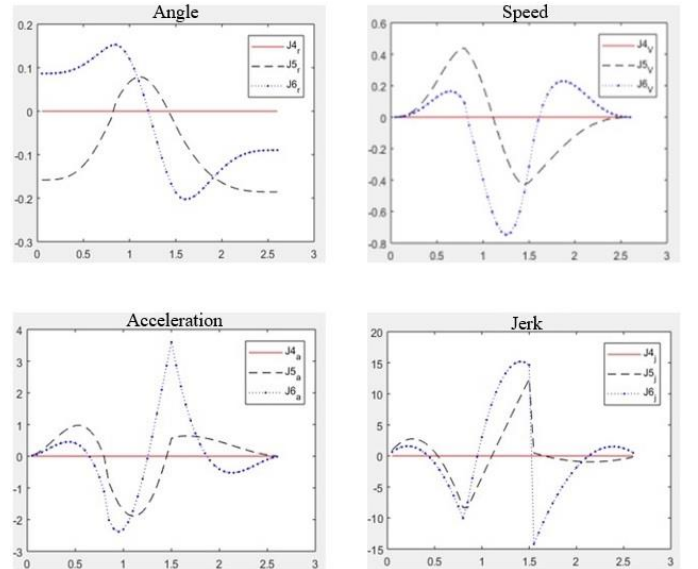
As shown in Figure 2, the optimal times for the robot to run three segments are 1.6447 seconds for the PSO algorithm, 1.6181 seconds for the GA-PSO algorithm, and 1.5481 seconds for the improved GA-PSO algorithm. It can be seen that the PSO algorithm has a fast search speed, but it cannot find the global optimal solution. It is prone to local optimal solutions. The GA-PSO algorithm increases the population diversity in the later stage based on the PSO algorithm, but its early optimization ability is not as good as the improved GA-PSO algorithm. The latter reduces the crossover and mutation probabilities in the early stage of iteration, fully exerting the speed advantage of the particle swarm, and increases the crossover and mutation probabilities in the later stage to increase the population diversity, giving it better optimization performance.

Applying the improved GA-PSO algorithm to the six-axis joints of the robot, with speed as π rad/s, acceleration as 10 rad/s²,

and jerk as 50 rad/s³ as constraints, the weight ratio of time to impact is $W_t:W_v=3:7$. The optimization of the six-axis three-segment time is performed, and the final result of the algorithm optimization iteration is shown in Figure 2, where the angles are in radians.



(a) Optimal curves for the first three axes: angle, speed, acceleration, and jerk



(b) Optimal curves for the last three axes: angle, speed, acceleration, and jerk

Fig. 3 Optimal Time Six-Axis Curves under Constraints

As shown in Figure 3, the six-axis speed, acceleration, and jerk are all within the constraints. The longest time of the six axes is selected for each segment, resulting in three times: 0.8395s, 0.7017s, and 1.1492s, with a total time of 2.6903s. The angle of the fourth axis remains unchanged, and there is a sudden change in jerk at the third key point, but the magnitude does not exceed 40 rad/s³, and the impact is small.

The optimal times for the three segments are substituted into Equations 5-7 to obtain the final trajectory of the six axes. The

trajectory of the robot hand end is shown in Figure 4, where (a) shows the original gesture trajectory of the robot end-space route, (b) shows the trajectory after impact optimization, which is a space curve, with a Z-axis offset of up to 5mm. The curve is mapped to the robot, and the Z-shape is restored by mirror reflection, and the path of the space curve is simulated in EPSON RC software C3 Sample, as shown in Figure 4(c).

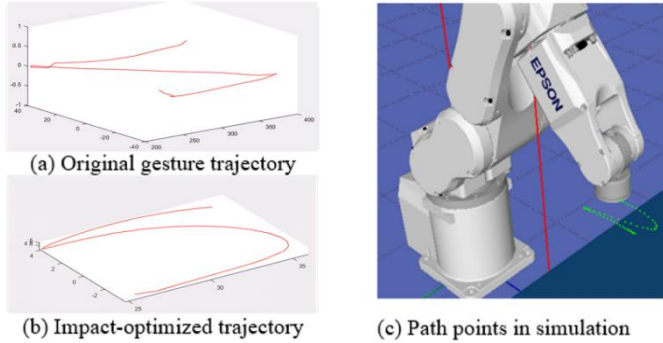


Fig. 4 end-effector trajectory of the robot arm

From the results in Figure 4, it can be seen that at the third point, sacrificing some impact to save time, in the joint space, the speed, acceleration, and jerk of the six axes are optimized to pass through the four key points in 2.69 seconds. The results indicate that the obtained trajectory is smooth and does not exceed the impact constraints. Increasing the number of key points can increase the matching degree with the original trajectory, but the impact optimization effect decreases. This paper takes four key points as an example, and the number of key points should be determined according to the actual situation.

A. Conclusion

This paper mainly studies the trajectory planning of industrial robots based on gesture recognition, using an improved GA-PSO algorithm to optimize the time-impact optimal trajectory. In the specific experiment, the EpsonC4 robot is used for simulation verification. Through optimization of the joint space coordinate points, the PSO algorithm, GA-

PSO algorithm, and improved GA-PSO algorithm are used to solve the problem. The experimental results show that the improved GA-PSO algorithm is superior to the PSO algorithm and the general GA-PSO algorithm in terms of convergence speed and optimal total time. Finally, the optimized trajectory is applied to the six-axis joints of the robot to ensure that the speed, acceleration, and jerk are within the constraint range, obtaining the optimal time-impact trajectory that satisfies the constraints.

ACKNOWLEDGMENT

Many thanks to the conference organizers, reviewers and friends for their support and help! We could not have successfully completed this conference paper without your support. We would like to express our heartfelt gratitude and deep respect!. This work was supported by the National Nature Science Foundation of China (No. 62171271).

REFERENCES

- [1] Dai Y, Xiang C, Zhang Y, et al. A Review of spatial robotic arm trajectory planning[J]. Aerospace, 2022, 9(7): 361.
- [2] Madridano A, Al-Kaff A, Martín D, et al. Trajectory planning for multi-robot systems: Methods and applications[J]. Expert Systems with Applications, 2021, 173: 114660.
- [3] Zhang T, Zhang M, Zou Y. Time-optimal and smooth trajectory planning for robot manipulators[J]. International Journal of Control, Automation and Systems, 2021, 19: 521-531.
- [4] Jin R, Rocco P, Geng Y. Cartesian trajectory planning of space robots using a multi-objective optimization[J]. Aerospace Science and Technology, 2021, 108: 106360.
- [5] Fernandes P B, Oliveira R C L, Neto J V F. Trajectory planning of autonomous mobile robots applying a particle swarm optimization algorithm with peaks of diversity[J]. Applied Soft Computing, 2022, 116: 108108.
- [6] Liu Y, Xiao F, Tong X, et al. Manipulator trajectory planning based on work subspace division[J]. Concurrency and Computation: Practice and Experience, 2022, 34(5): e6710.
- [7] Yu X, Dong M, Yin W. Time-optimal trajectory planning of manipulator with simultaneously searching the optimal path[J]. Computer Communications, 2022, 181: 446-453.